

# L4\_L3 Extension - Report

1. Group Member(s): Max De Jesus

2. GH Repo: [https://github.com/theMaxiim/471c/tree/concurrency\\_extension](https://github.com/theMaxiim/471c/tree/concurrency_extension)

## 3. Deviations from Proposal:

1. None, all features were implemented, albeit, with certain limitations. For instance, given the approach of cooperative concurrency, which leverages jump/labels, certain design decisions needed to be considered. Specifically, channels only support a single parked sender and a single parked receiver at a time, given that the channel stores one continuation slot rather than a queue of waiters (as in other prod language implementations), however, it serves as a suitable implementation for a PoC and passes the tests outlined in our "Success Metrics" section of our proposal. Multiple goroutines ("spawn" in L4) sending to the same channel is not supported, though, we can work around this by using multiple channels per sender. Additionally, for the purposes of this PoC, buffered channels are limited to a capacity of one, as a dynamic indexing into heap-allocate arrays are not possible with our current implementation of Load/Store indices.

Lastly, of course, since our control transfer is cooperative through the use of Jump/Label, it only occurs explicitly at send/recv/spawn points.

**Note:** a group of example programs is stored within L4/examples, they serve as E2E tests to validate correct behavior.

**CI:** Code coverage is @ 100% for L4 (on concurrency\_extension branch)

## 4. Course and Extension Reflection:

1. The implementation of this language extension was one that, although quite difficult, was, ultimately, extremely rewarding. Through it, I reinforced the patterns that Professor Clause has instilled in his students throughout the semester: extensive testing, abstraction as a key mechanism, and draw inspiration from those before you. Specifically, on that latter point, the L4 extension would not have been possible without, first, the direction from Dr. Clause, which led to uncovering an existing implementation of cooperative scheduled fibers (i.e VM initiated "threads") in Wren. This language manages continuations through what it calls a "trampoline scheduler," where, each spawn creates a fiber, each blocking operation yields to the next, and execution "ping-pongs" between fibers until all is complete.

This particular pattern inspired the implementation of go-like concurrency in the L4/L3 extension, albeit, to a lesser degree given the course scope.

Given its immense applicability in the Software Engineering field, I find it critical to reiterate the importance of unit tests, which are, indubitably, a key takeaway from CISC471, given the explicit dependence in each language layer. Although they may, at times, be painful to implement, they are the stepping stone that unlock abstraction to work as intended. When done correctly, they serve as a layer of trust, which allows for your program to become, not a stochastic black box, but a dependable unit that can later serve part of a much larger system.